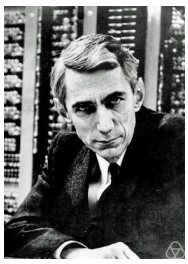
TEL354 - Minería de datos

Clase 16: Neural Language Models

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Teoría de Shannon

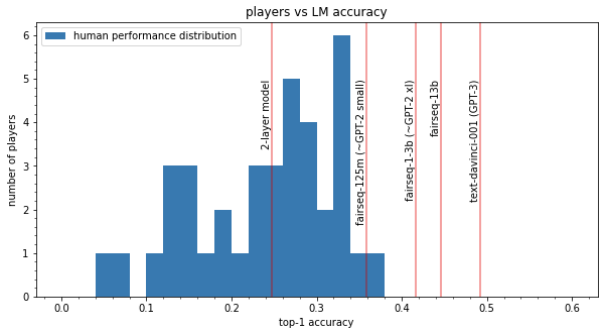
Shannon fundó la **teoría de la información** en 1948, lo que permite cuantificar y medir la cantidad de información en un mensaje y la incertidumbre asociada a este. Introdujo el concepto de "entropía" informacional.



Además, mostró cómo predecir lo que viene a continuación en una secuencia, lo cual es exactamente lo que hacen los LLMs en la actualidad, basan sus respuestas en la probabilidad de las palabras siguientes en función de los datos previos.

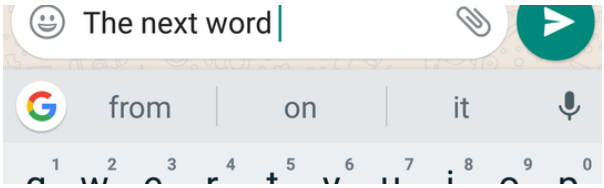
Experimento next-token prediction

¿Son mejores o peores los modelos de lenguaje en comparación a los humanos para predecir la siguiente palabra?



¿Qué es un modelo de lenguaje?

Un modelo de lenguaje es básicamente una distribución de probabilidad sobre palabras o secuencias de palabras. Entre otras cosas, pueden sugerir la siguiente palabra que escribimos.



Probabilistic Language Model

Dada una secuencia de palabras D en una oración, la tarea es calcular las probabilidades de todas las palabras que terminarían esta oración. Las palabras se eligen de un vocabulario dado (V). Es decir, se necesita calcular la siguiente probabilidad condicional:

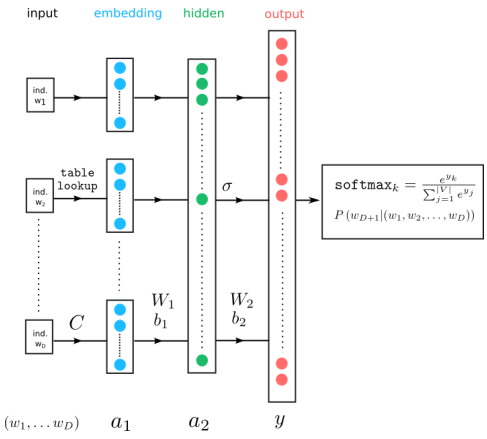
$$P\left(w_{D+1}^{(i)}|(w_1^{(i)}, w_2^{(i)}, \ldots, w_D^{(i)})\right)$$

para cualquier ejemplo dado (i). Cada palabra está representada por un vector w, que tiene la dimensión del tamaño del diccionario ($|V|$). Por ejemplo, considere: "I am happy because I am learning".

$$P(\text{happy}|\text{I am}) = \frac{C(\text{I am happy})}{C(\text{I am})} = \frac{1}{2} = 0.5$$

Neural Language Model

La capa de entrada contiene los índices de las palabras del vocabulario y hay D de ellas (ya que un ejemplo de entrenamiento tiene D palabras, y la tarea es predecir la palabra $D+1$ para terminar la oración).



La segunda capa es la capa de embedding y transforma cada palabra de entrada (que tiene la dimensión del vocabulario $|V|$), en una representación vectorial de menor dimensión (digamos h_1).

Hay una capa oculta con tamaño h_2, y la capa de salida tiene tamaño $|V|$ que luego se introduce en una función softmax para calcular la probabilidad de cada palabra en el vocabulario.

Datos en secuencia

Supongamos que tiene una fotografía de la posición de una pelota en el tiempo T y desea predecir la dirección que se moverá en $T+1$.



Con esta información, cualquier respuesta sería adivinar al azar.

Se necesita una secuencia de fotografías con las posiciones de la pelota antes del tiempo T para tener suficiente información y poder predecir la dirección en $T+1$.

Esto es una secuencia, un orden particular en que una cosa sigue a otra.

Memoria Secuencial

¿Qué tan difícil es nombrar el alfabeto sin mirarlo?

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Debería ser bastante fácil, ¿verdad? Si ya aprendieron esta secuencia específica antes, debería aparecer relativamente rápido y ser capaz de recitarlo.

Ahora trate de decir el alfabeto al revés.

Z Y X W V U T S R Q P O N M L K J I H G F E D C B A

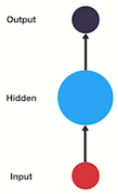
Apuesto a que esto es mucho más difícil. A menos que haya practicado esta secuencia específica antes, es probable que tenga dificultades.

Hay una razón muy lógica por la que esto puede ser difícil. Usted aprendió el alfabeto como una secuencia. La memoria secuencial es un mecanismo que facilita que su cerebro reconozca patrones que siguen una secuencia.

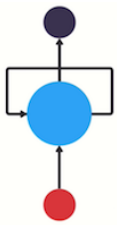
Recurrent Neural Networks (RNNs)

Replican el concepto de memoria secuencial.

¿Cómo agregar memoria a una red Feed Forward tradicional?



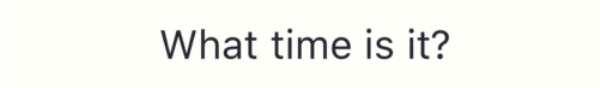
¿Qué tal si agregamos un bucle en la red neuronal que pueda transmitir información previa hacia adelante?



Una RNN tiene un mecanismo de bucle que actúa como una autopista para permitir que la información fluya de un paso al siguiente.

¿Cómo funciona el proceso en una RNN?

Supongamos un usuario pregunta:



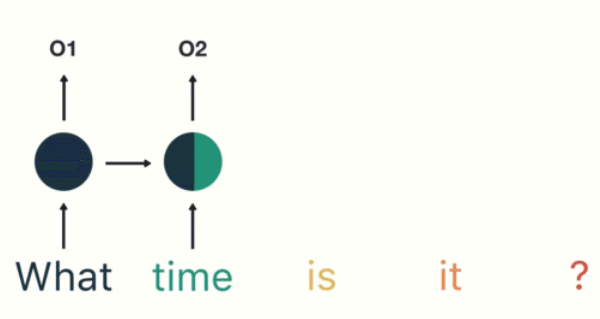
Para empezar, dividimos la oración en palabras individuales. El trabajo de RNN es secuencial, por lo que lo alimentamos una palabra a la vez.



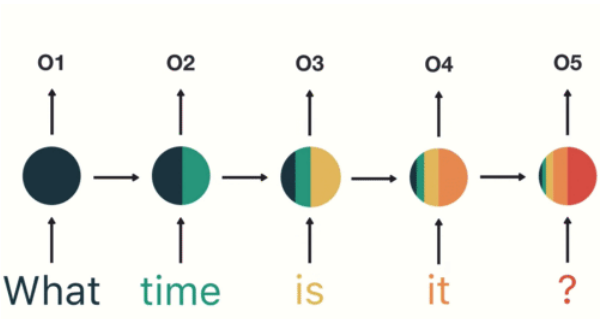
El primer paso es introducir "What" en la RNN, la cual codifica la palabra y produce una salida.



Para el siguiente paso, se alimenta la red con la palabra "time" y el estado oculto del paso anterior. La RNN ahora tiene información de las palabras “What” y “time”.



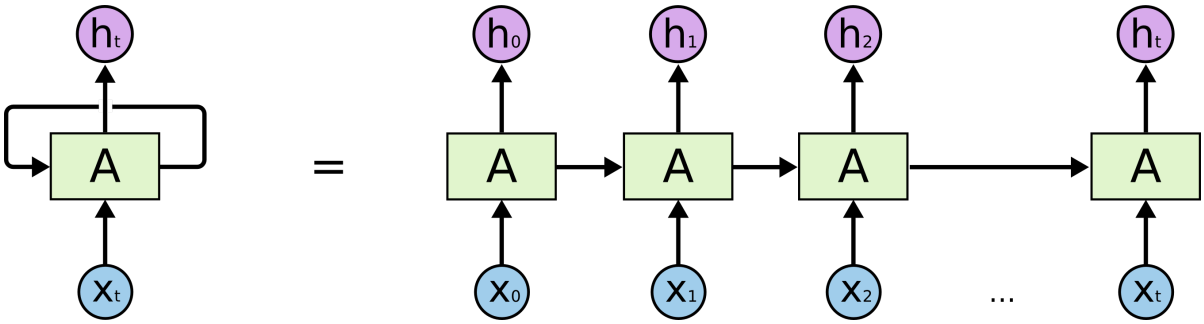
Se repite este proceso, hasta el paso final. Puede ver en el paso final que la RNN ha codificado información de todas las palabras en los pasos anteriores.



Dado que el output final se creó a partir del resto de la secuencia, podemos tomar el output final y pasarlo a la capa feed-forward para clasificar la intención.

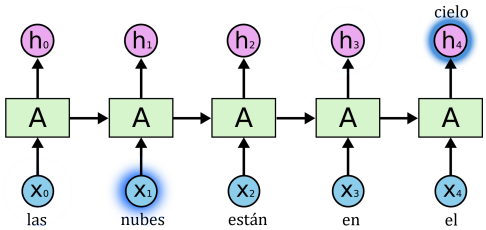


Una red neuronal recurrente desenrollada



Dependencias a corto plazo

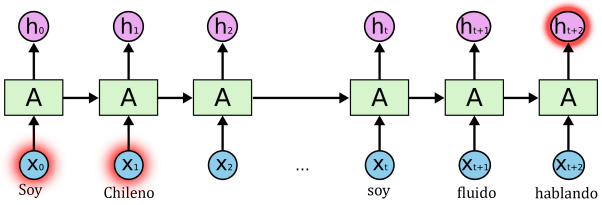
Por ejemplo, considere un modelo de lenguaje que intente predecir la siguiente palabra en función de las anteriores.



Si estamos tratando de predecir la siguiente palabra en "las nubes están en el", no necesitamos tanto contexto: es bastante obvio que la siguiente palabra será "cielo". En tales casos, donde la brecha entre la información relevante y el lugar donde se necesita es pequeña, las RNN pueden aprender a usar la información pasada.

El problema de las dependencias a largo plazo

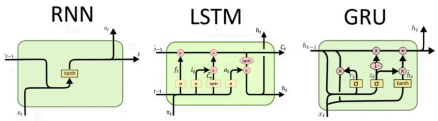
Pero también hay casos en los que necesitamos más contexto. Considere tratar de predecir la siguiente palabra en el texto "... soy fluido hablando".



Desafortunadamente, a medida que crece esa brecha, las RNN se vuelven incapaces de aprender a conectar la información.

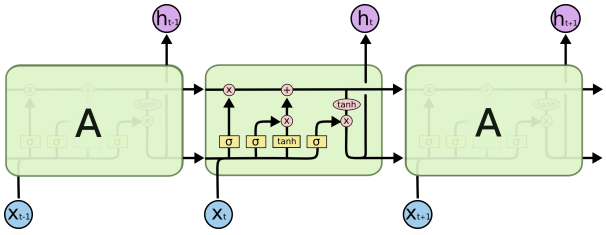
Redes LSTM y GRU

- Debido al problema de las gradientes desvanecientes, las RNNs sufren de memoria a corto plazo y pueden "olvidar" las primeras palabras de la secuencia.
- Para mitigar la memoria a corto plazo, se crearon dos redes neuronales recurrentes especializadas: Long Short-Term Memory (LSTM) y Gated Recurrent Units (GRU).
- LSTM y GRU son capaces de aprender secuencias de términos a largo plazo utilizando mecanismos llamados "*gates*". Estas "puertas" son diferentes operaciones tensoriales que pueden aprender qué información mantener y cual eliminar en la capa oculta.

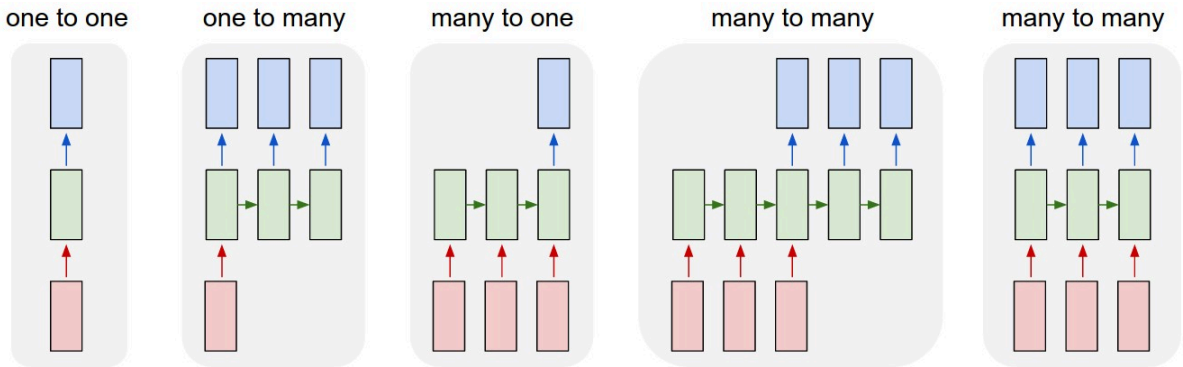


LSTM

- Las redes *Long Short Term Memory* (memoria a largo plazo), generalmente llamadas simplemente "LSTM", son un tipo especial de RNN, capaces de aprender dependencias a largo plazo.
- Cada unidad mantiene un estado de memoria de la celda c_t en el tiempo t.
- Tiene 3 compuertas: compuerta de olvido, compuerta de entrada y compuerta de salida.



Tipos de redes neuronales recurrentes (RNN)



In [1]:
```
import tensorflow as tf
from tensorflow import keras
import numpy as np
```

In [2]:
```
with open("data/quijote.txt", encoding="utf8") as f:
    text = f.read()
```

In [3]:
```
print(text[:500])
```

```
DON QUIJOTE DE LA MANCHA
Miguel de Cervantes Saavedra

PRIMERA PARTE
CAPÍTULO 1: Que trata de la condición y ejercicio del famoso hidalgo D. Quijo
te de la Mancha
En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tie
mpo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín
flaco y galgo corredor. Una olla de algo más vaca que carnero, salpicón las m
ás noches, duelos y quebrantos los sábados, lentejas los viernes, algún palom
ino de añadidura los
```

In [4]:
```
"".join(sorted(set(text.lower())))
```

Out[4]:
```
'\n !"\'(),-.0123456789:;<?[]abcdefghijklmnopqrstuvwxyz¡«»¿—‘’“”'
```

In [5]:
```
tokenizer = keras.preprocessing.text.Tokenizer(char_level=True)
tokenizer.fit_on_texts(text)
```

In [6]:
```
max_id = len(tokenizer.word_index) # numero de caracteres distintos
max_id
```

Out[6]:
```
65
```

In [7]:
```
dataset_size = tokenizer.document_count # numero total de caracteres
dataset_size
```

Out[7]:
```
1038397
```

In [8]:
```
[encoded] = np.array(tokenizer.texts_to_sequences([text])) - 1
train_size = dataset_size * 90 // 100
dataset = tf.data.Dataset.from_tensor_slices(encoded[:train_size])
```

In [9]:
```
n_steps = 100
window_length = n_steps + 1 # target = input shifted 1 character ahead
dataset = dataset.window(window_length, shift=1, drop_remainder=True)
```

In [10]:
```
dataset = dataset.flat_map(lambda window: window.batch(window_length))
```

In [11]:
```
batch_size = 32
dataset = dataset.shuffle(10000).batch(batch_size)
dataset = dataset.map(lambda windows: (windows[:, :-1], windows[:, 1:]))
```

In [12]:
```
dataset = dataset.map(
    lambda X_batch, Y_batch: (tf.one_hot(X_batch, depth=max_id), Y_batch))
```

In [13]:
```
dataset = dataset.prefetch(1)
```

In [14]:
```
for X_batch, Y_batch in dataset.take(1):
    print(X_batch.shape, Y_batch.shape)
```

```
(32, 100, 65) (32, 100)
```

Configurar el modelo

In [15]:
```
model = keras.models.Sequential([
    keras.layers.GRU(128, return_sequences=True, input_shape=[None, max_id],
                     dropout=0.2),
    keras.layers.GRU(128, return_sequences=True,
                     dropout=0.2),
    keras.layers.TimeDistributed(keras.layers.Dense(max_id,
                                                    activation="softmax"))
])
```

In [16]:
```
model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")
```

In [25]:
```
from tensorflow.keras.optimizers.legacy import Adam
model.compile(loss="sparse_categorical_crossentropy", optimizer=Adam())
```

Entrenar el modelo

Con el Quijote se demora varias horas en entrenar. Puede cargar los pesos y evitar el entrenamiento o bien usar menos datos para entrenar.

In [26]:
```
model.load_weights("weights/cp.ckpt")
```

Out[26]:
```
<tensorflow.python.checkpoint.checkpoint.CheckpointLoadStatus at 0x7f7c7d57f7
00>
```

In [ ]:
```
checkpointer = tf.keras.callbacks.ModelCheckpoint(filepath='weights/cp.ckpt',
                                save_weights_only = True, verbose = 0)
```

In [ ]:
```
history = model.fit(dataset, epochs=10, callbacks=[checkpointer])
```

Usar el modelo para generar texto

In [18]:
```
def preprocess(texts):
    X = np.array(tokenizer.texts_to_sequences(texts)) - 1
    return tf.one_hot(X, max_id)
```

In [19]:
```
def next_char(text):
    X_new = preprocess([text])
    y_proba = model(X_new)[0, -1:, :]
    rescaled_logits = tf.math.log(y_proba)
    char_id = tf.random.categorical(rescaled_logits, num_samples=1) + 1
    return tokenizer.sequences_to_texts(char_id.numpy())[0]
```

In [20]:
```
next_char("don qui")
```

Out[20]:
```
'j'
```

In [28]:
```
def complete_text(text, n_chars=1):
    for _ in range(n_chars):
        text += next_char(text)
    return text
```

In [33]:
```
complete_text("don qui", 300)
```

Out[33]:
```
'don quijote de lo que mo parece, ni lo soy envidio.\nines fama confusa y suc
eso. no hen veosado mano amante, si no está admiración de don quijote, que se
laro pasaba las encantaderas de la locura de tus leeguos, sino a que hubleran
en la tierra engañada, porque no fueran paso, si parece, que aun mesma hec'
```

In [22]:
```
complete_text("don qui", 25)
```

Out[22]:
```
'don quijote de poco semejante, l'
```

In [23]:
```
complete_text("don qui", 25)
```

Out[23]:
```
'don quijote, no fueron los cildi'
```

In [24]:
```
complete_text("don qui", 25)
```

Out[24]:
```
'don quifote de librar e reposado'
```